MEDIUM-2：计算机中的整数和浮点数

Step1：甜度与温度传感器：补码与隐式类型转换

【任务 1.1：到底有多甜？】

1.按**无符号整数**解析，奶茶的甜度是：156。

2.按**有符号整数**解析，甜度值是：-100。

3.为什么底层用**补码**?

原码：最高位当符号位，其余为绝对值。**反码**：负数按位取反。**补码**：负数按位取反加一

补码的核心好处我认为有三个：补码的零是唯一的，补码的**加减法统一**（根据模2^n同余的数学性质推导），8，16，32字节符号扩展时不出错。

【任务 1.2：冰块引发的火灾警报】

输出：[警告!] 温度过高！触发消防喷淋系统!

解释：当两个不同类型的数进行运算时，会自动向范围更大的转换（**隐式类型转换**）。原理：1.整数提升，2.有符号的值不能完全表示无符号的值，有符号转无符号。反之则无符号转有符号。

所以：老板，写程序要注意，无符号有符号混用要小心!

Step2：反物质珍珠 —— 整数溢出与二进制下的运算

【任务 2.1：算术溢出的惨剧】

1.保存的二进制位模式是：10011001

2.系统认为有：-103颗珍珠。这是正溢出。

3.

```
 // 若相加不发生溢出返回 1，发生溢出返回 0
int is_safe(int x, int y) {

    int sum = x + y;
    int x_sign = (x >> 31) & 1;
    int y_sign = (y >> 31) & 1;
    int sum_sign = (sum >> 31) & 1;
    int result;
    result=1;
    // 请补全代码写在TO DO与END OD TO DO之间：
    //TO DO
    if(x_sign==1&&y_sign==1){
        if(sum_sign==0){
            result=0;
        }
    }
    if(x_sign==0&&y_sign==0){
    if(sum_sign==1){
        result=0;
    }
```

```
}
    //END OF TO DO

    return  result;
}
```

【任务 2.2：没有乘法器的悲哀】

```
int final_milk(int x) {

    int y;

    // 请补全代码写在TO DO与END OD TO DO之间：
    //TO DO
    y=(x<<5)-(x<<2)+x;

    //END OF TO DO

    return y;
}
```

小问题：如果x往右移五位直接变成负数怎么办（用unsigned），但如果直接截断那就废了。

Step3：丝滑配比阀 —— 认识IEEE 754标准下的浮点数

【任务 3.1：对学长提示的深入理解】

1.float：符号位（sign）1位，阶码（exponent）：8位，尾数（fraction）：23位。

double：（sign）1位，（exponent）：11位，（fraction）：52位。

2.偏置值=$(2^{k-1}-1)$，其中k等于指数域位数（8or11）。意义：**方便比较大小，保留特殊编码**

解释：**比较大小**：当需要比较阶码时，直接按位比较（无进位链，符号判断等，速度快），不需要加减，也就没有任何溢出的风险，并且逻辑更直观。**特殊值编码**：全零（转化为非规格数或零的标志），全一刚好落在两端，其中，全一留出来表示NaN（阶码全一，尾数非零）和无穷（阶码全一，尾数为零）。

3.非规格化数的特点：这个设计很巧妙，加上偏置后，阶码（exponent）全零时，已经是一个非常小的数（float：约`2^-126`）此时为了表示更小的数（设计上是**向零平稳过渡**）就默认最前面的**"整数"由一变为零**，然后实际上真正的有效数字从**尾数（fraction）**中开始，所以非规格数牺牲了一定的有效精度，为了表示出更小的数。**示意图如下**

```
text                                                    复制  下载

0
|
|  2^-149
|  2 × 2^-149
|  3 × 2^-149
|  ...
|  (1 - 2^-23) × 2^-126   ← 最大非规格化数
|  1.0 × 2^-126           ← 最小规格化数
|  1.000...0001 × 2^-126
|  ...
|  2.0 × 2^-126
|  ...
→ 数值增大
```

4.阶码全为1且尾数全为0：无穷。阶码全为1尾数不为零：NaN。

特别注意：已阅读《深入理解计算机系统》了解浮点数

【任务 3.2：补全浮点数对照表】

二进制位模式	编码类型 (规格化 / 非规格化 / 特殊值)	实际阶码 e (十进制)	尾数真值 M (二进制小数)	十进制最终真值
0 0110 100	规格化	6−7=−1	1.100（2）=1.5	0.75
0 0000 011	**(A) *非规格化***	**(B) -6**	**(C) 0.011（2）=0.375**	**(D) *0.375*2^-6***
0 1111 000	**(E) *特殊值***	不适用	不适用	+∞+∞ (正无穷)
0 1001 010	规格化	**(F) *2***	**(G) *1.010（2）=1.25***	**(H) *5***

1.最小的正非规格数二进制位模式：`0 0000 001`,对应的十进制数是2^-9。最大是`0 0000 111`,对用十进制数是0.875*2^-6。

2.最大的是`0 1110 111`，对应的十进制数是1.875*2^7。最小`0 0001 000`，十进制数2^-6

3.非规格化数的实际阶码取 1−Bias，是为了和最小规格化数共用同一个指数（**“整数项由一变为零，无形中阶码移动一位”**），让尾数从 0.111…… 到 1.000…无缝衔接，实现渐进下溢。如果取 −Bias，中间会漏掉一大段数，不连续。

Step4：消失的营业额 —— 浮点加法、舍入与结合律

【任务 4.1：设计加法程序】

1.为什么**小阶向大阶看齐**？小阶向大阶看齐要右移（**为扩大阶码准备**），反之左移。右移通常损失的是尾数末端的低位信息。左移如果超出尾数可用的位数，则可能破坏高位的有效信息。后者通常更加不利。并且右移造成的精度损失，则可以通过保护位、舍入等机制尽可能减小。（左移不知道有没有这种机制）

2.**四种**可选的舍入处理方法：

舍入方式	核心规则
舍入到最近值，平局取偶数	选择距离精确结果最近的可表示值；恰好等距时，选择末位为偶数的那个
向零舍入	丢弃超出精度的部分，不向远离零的方向进位
向正无穷舍入	选择不小于精确结果的最小可表示值
向负无穷舍入	选择不大于精确结果的最大可表示值

3.八位迷你浮点系统：

```
#include <stdio.h>
int main(){
    char count1[9];
    char count2[9];
    scanf("%8s %8s",count1,count2);
```

```
    unsigned char a=0,b=0;
    for(int i=0;i<7;i++){
        a|=count1[i]-'0';
        a<<=1;
        b|=count2[i]-'0';
        b<<=1;
    }
    unsigned char exponentA=a>>3;
    unsigned char exponentB=b>>3;
    unsigned char significandA=8|(a&7);
    unsigned char significandB=8|(b&7);
    unsigned char exponent=0;
    int r;
if (exponentA >= exponentB) {
    r = exponentA - exponentB;
    significandB >>= r;
    exponent = exponentA;
} else {
    r = exponentB - exponentA;
    significandA >>= r;
    exponent = exponentB;
}
    unsigned int t=significandA+significandB;
    if(t>=16){
        t>>=1;
        exponent+=1;
    }
    unsigned char fraction=t&7;
    unsigned char result=0;
    result|=(exponent<<3)|fraction;
for (int i = 7; i >= 0; i--) {
    printf("%u", (result >> i) & 1u);
}
printf("\n");
    return 0;
}
```

【任务 4.2：老板为什么每天都在亏钱？】

核心原因：浮点数的精度有限制！下面以float类型举例。float中的fraction为23位，将二进制能表示的所有值对应到十进制：

$$2^x = 10^y,\ 2^{24} = 16777216 = 1.6777216 \times 10^7$$

说明float型数的有效精度为七位有效数字，而老板的money居然有10^{20}

因此，老板的21位数字，float所能记住的大概只有7位，后面的位只能舍入。因此被改造成

$$1e20f \approx 1.0000000200408773 \times 10^{20}$$

这就是所谓的**精度丢失。**

数学原理如下：

$$10^{20} \approx 1.3552527156068805 \times 2^{66}$$

float 尾数只能保留 23 位：

$$0.3552527156068805 \times 2^{23} \approx 2980075.77$$

四舍五入：

$$2980075.77 \to 2980076$$

实际尾数：

$$1 + \frac{2980076}{2^{23}} = 1.355252742767334$$

实际值：

$$1.355252742767334 \times 2^{66} = 100000002004087734272 \approx 1.0000000200408773 \times 10^{20}$$

所以可以解释15块钱凭空消失的原因（**大数吸收**）：15元太少了，在**对阶**的过程中15（1.875*2^3）要右移63位才能与-1e20f的**指数对齐**，直接变成0了。daily_expense加上15后根本不会影响结果。而初始资产与支出的净值是相反数，相加后直接变为0，0与15.0进行运算则正常进行。

所以，老板，这15 块不是丢了，是被浮点数的精度限制吞掉了。